

Project 1: Regression Analysis, Resampling Methods and Gradient Descent

Teresa Macarulla Uriarte

FYS-STK3155 Applied Data Analytics and Machine Learning
University of Oslo

(Dated: October 5, 2026)

Polynomial regression of noisy samples of Runge’s function was used to study model complexity, regularisation, resampling, and gradient-based optimisation. Ordinary Least Squares (OLS), Ridge, and Lasso regression were compared using training/test analysis, bootstrap resampling, and K -fold cross-validation. Increasing the sample size allowed more complex polynomial models to be fitted before variance became dominant, whereas increasing the noise level reduced the useful model complexity. Bootstrap resampling confirmed the expected transition from bias-dominated underfitting to variance-dominated over-fitting, while cross-validation provided a more stable basis for model selection. For $n = 200$ and $\sigma = 0.1$, final five-fold cross-validation selected OLS with degree $d = 14$ and a mean MSE of 0.011852, compared with 0.012489 for Ridge and 0.013282 for Lasso. Ridge and Lasso nevertheless showed substantially greater stability at excessive polynomial degree. Analytical and automatically differentiated gradients agreed to machine precision, and gradient descent reproduced the closed-form OLS and Ridge solutions. Momentum gave the fastest full-batch convergence among the tested optimisers, while mini-batch stochastic optimisation offered no computational advantage for the small data set considered here. Overall, the results show that regularisation primarily improves robustness rather than the minimum prediction error for this particular regression problem.

CONTENTS

I. Introduction	1	V. Conclusions and Perspectives	18
II. Theory	2	A. Use of AI/LLM Tools	18
A. Polynomial regression and performance measures	2	1. Tools Used	18
B. Ridge and Lasso regularisation	2	2. Writing and Editing	18
Ridge regression	2	3. Programming Assistance	18
Lasso regression	3	References	19
C. Bias–variance decomposition and resampling	3		
D. Gradient-based optimisation	4		
E. Momentum and adaptive optimisation methods	4		
F. Stochastic gradient descent	4		
III. Methods and Implementation	5		
A. Data generation and preprocessing	5		
B. Regression and optimisation implementations	5		
OLS implementation and experimental setup	5		
Ridge implementation and experimental setup	5		
Gradient descent and AD	6		
Momentum and adaptive optimizer comparison	6		
Lasso implementation and validation	6		
Stochastic gradient descent	6		
C. Bootstrap bias–variance analysis	7		
D. Cross-validation and comparison	7		
IV. Results and Discussion	8		
A. OLS and Ridge regression	8		
OLS results	8		
Ridge results	8		
B. Bias–variance analysis and cross-validation	9		
Bias–variance analysis	9		
Cross-validation and comparison of model assessment	10		
C. Gradient descent, adaptive methods, Lasso and SGD	12		
Plain gradient descent and AD	12		
Momentum and adaptive optimizers	13		
Lasso regression	13		
Stochastic gradient descent	14		
D. Final model comparison	16		

I. INTRODUCTION

Regression analysis provides a controlled setting for studying some of the central questions of machine learning: how complex a model should be, how sensitive its predictions are to the available data, and how well its performance on unseen observations can be estimated. Polynomial regression is particularly useful for illustrating these issues because increasing the polynomial degree improves the flexibility of the model, but may also lead to poorly conditioned fits and large sensitivity to noise. In this project, Runge’s function is used as a simple one-dimensional test problem to study these effects in a setting where the underlying function is known [1].

The main objective is to investigate how polynomial model complexity, sample size, noise, and regularisation affect prediction and generalisation. Ordinary Least Squares (OLS) is used as the unregularized reference, while Ridge and Lasso regression are introduced to examine two different ways of controlling model complexity. Bootstrap resampling and cross-validation are used to study the bias–variance trade-off and to distinguish genuine predictive performance from behaviour that depends on a particular train–test split. A second aim

is to study the numerical optimisation of the same regression problems, comparing closed-form solutions with gradient-based methods and examining the effects of different parameter-update rules and stochastic mini-batch gradients.

The remainder of the report first introduces the theoretical background, followed by the numerical methods and implementation. The resulting regression, resampling, and optimisation studies are then presented and discussed, before the three regression methods are brought together in a final cross-validated model comparison. The report concludes with the main findings and possible extensions.

All code developed for this work is available at this [GitHub](#) repository. The implementation was divided into three source files. The main experimental workflow and the individual experimental parts are defined in `main.py`; data generation, regression, resampling, and optimization routines are implemented in `utilities.py`; and all figure-generation routines are collected in `plot_generator.py`. This separation was used so that the numerical methods could be tested independently from the plotting and experiment-control code.

II. THEORY

A. Polynomial regression and performance measures

A regression problem can be formulated by assuming that the observed targets are generated by an underlying function with additive noise,

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \quad (1)$$

where the noise terms are assumed to be independent. In this project, the underlying function is Runge's function,

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1], \quad (2)$$

to which Gaussian noise is added to generate the regression data [1]. The objective is to construct an approximation $\tilde{y}$ to the unknown relation between x and y [2].

For a polynomial model of degree d , the approximation is written as a linear model in the parameters,

$$\tilde{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}, \quad (3)$$

where $\boldsymbol{\theta} = (\theta_0, \dots, \theta_d)^T$ and the polynomial design matrix is

$$\mathbf{X} = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^d \\ 1 & x_1 & x_1^2 & \cdots & x_1^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^d \end{pmatrix}. \quad (4)$$

Thus, although the model is nonlinear in the input x , it is linear in the parameters $\boldsymbol{\theta}$. Increasing d increases the

flexibility of the model, but also makes the polynomial design matrix increasingly susceptible to ill-conditioning [2].

Ordinary Least Squares (OLS) determines the parameters by minimizing the mean squared residual,

$$C(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2. \quad (5)$$

Setting the gradient of this cost function to zero gives the normal equations $\mathbf{X}^T \mathbf{X} \boldsymbol{\theta} = \mathbf{X}^T \mathbf{y}$ and, when $\mathbf{X}$ has full column rank,

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}. \quad (6)$$

This expression is the analytical OLS solution, although in numerical implementations the pseudoinverse or an SVD-based (Singular Value Decomposition) solution is preferable to explicitly forming the inverse of $\mathbf{X}^T \mathbf{X}$, particularly for high-degree polynomial models [2].

The quality of the predictions is quantified using the Mean Squared Error (MSE) and the coefficient of determination R^2 ,

$$\text{MSE} = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2, \quad (7)$$

$$R^2 = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=0}^{n-1} y_i. \quad (8)$$

The MSE measures the average squared prediction error, whereas R^2 compares the model with the constant prediction $\bar{y}$ [2].

Finally, preprocessing must be handled consistently. OLS predictions are invariant under a rescaling of the feature columns, although centering and scaling can improve numerical conditioning and become essential for the regularized methods considered later. Centering the predictors and targets also provides a convenient way of handling the intercept. Importantly, all means and scaling factors must be computed from the training data only and then applied unchanged to the test data, since preprocessing the complete data set before the split would introduce information leakage [1; 2].

B. Ridge and Lasso regularisation

Ridge regression

Ridge regression extends OLS by penalizing large model coefficients. Using the normalized cost-function convention adopted in this work,

$$C_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \lambda \boldsymbol{\theta}^T \mathbf{P} \boldsymbol{\theta}, \quad (9)$$

where $\lambda \geq 0$ controls the strength of the regularisation. The corresponding closed-form estimator is

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + n\lambda \mathbf{P})^{-1} \mathbf{X}^T \mathbf{y}, \quad (10)$$

with OLS recovered for $\lambda = 0$ and where $\mathbf{P} = \text{diag}(0, 1, \dots, 1)$ so that the intercept is not regularized. Ridge therefore introduces bias in exchange for a reduction in parameter variance and improved stability in directions that are weakly constrained by the data [2].

This effect is particularly transparent in the singular-value basis. For a singular value σ_i of $\mathbf{X}$, the corresponding contribution to the fitted values is multiplied by the shrinkage factor

$$s_i(\lambda) = \frac{\sigma_i^2}{\sigma_i^2 + n\lambda}. \quad (11)$$

Directions with $\sigma_i^2 \gg n\lambda$ are therefore almost unaffected, whereas directions with $\sigma_i^2 \ll n\lambda$ are strongly suppressed. Ridge consequently damps most strongly the poorly determined singular-value modes that contribute most to the variance of an unregularized OLS fit [2].

Lasso regression

Lasso regression replaces the 2-norm Ridge penalty with an 1-norm L_1 penalty (last element in Eq. (9) and in Eq. (12) respectively). Using the same normalized cost convention as for Ridge,

$$C_{\text{Lasso}}(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \lambda \sum_{j=1}^{p-1} |\theta_j|, \quad (12)$$

where the intercept θ_0 is not regularized. In contrast to Ridge, the L_1 penalty can set individual coefficients exactly to zero [2].

The absolute-value penalty is not differentiable at $\theta_j = 0$. Its derivative must therefore be replaced by the sub-differential

$$\partial|\theta_j| = \begin{cases} \{-1\}, & \theta_j < 0, \\ [-1, 1], & \theta_j = 0, \\ \{+1\}, & \theta_j > 0. \end{cases} \quad (13)$$

The soft-thresholding operator (Eq. (14)) expresses the behaviour of Lasso regularisation and forms the basis of coordinate-descent algorithms.

$$S_\gamma(z) = \text{sign}(z) \max(|z| - \gamma, 0), \quad (14)$$

If the coefficient is large enough, $|z| > \gamma$, the operator reduces its magnitude. If the coefficient is smaller than the threshold $|z| \leq \gamma$, it becomes exactly zero.

C. Bias–variance decomposition and resampling

Consider observations generated according to Eq. 1, where $f(\mathbf{x})$ is the deterministic underlying function. A

model fitted to a random training set produces predictions $\tilde{\mathbf{y}}$. Because the training set changes from one realization to another, $\tilde{\mathbf{y}}$ is itself a random quantity. For fixed input locations, the expected prediction error averaged over the data points is

$$\frac{1}{n} \sum_{i=0}^{n-1} \mathbb{E} [(y_i - \tilde{y}_i)^2]. \quad (15)$$

Using $y_i = f_i + \epsilon_i$, and adding and subtracting $\mathbb{E}[\tilde{y}_i]$,

$$y_i - \tilde{y}_i = (f_i - \mathbb{E}[\tilde{y}_i]) + (\mathbb{E}[\tilde{y}_i] - \tilde{y}_i) + \epsilon_i. \quad (16)$$

After squaring and taking the expectation, the cross terms vanish because $\mathbb{E}[\epsilon_i] = 0$ and $\mathbb{E}[\tilde{y}_i - \mathbb{E}[\tilde{y}_i]] = 0$. Therefore,

$$\begin{aligned} \frac{1}{n} \sum_i \mathbb{E} [(y_i - \tilde{y}_i)^2] &= \frac{1}{n} \sum_i \underbrace{(f_i - \mathbb{E}[\tilde{y}_i])^2}_{\text{Bias}^2} \\ &\quad + \underbrace{\frac{1}{n} \sum_i \mathbb{E} [(\tilde{y}_i - \mathbb{E}[\tilde{y}_i])^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Noise}}. \end{aligned} \quad (17)$$

This is the bias–variance decomposition of the prediction error [3]. The squared bias measures the systematic difference between the true function and the average prediction over possible training sets, whereas the variance measures how strongly the fitted prediction changes when the training sample is changed. The final term, σ^2 , is the irreducible error due to the observation noise.

In practice, the expectation over possible training sets is not directly available from a single observed data set. Bootstrap resampling approximates this ensemble by repeatedly drawing new training samples, with replacement, from the observed training data and fitting the model to each bootstrap replica [3].

Cross-validation provides an alternative estimate of the generalisation error without repeatedly generating resampled data sets. In K -fold cross-validation, the available observations are partitioned into K subsets. For each fold k , the model is fitted using the remaining $K - 1$ folds and evaluated on the held-out fold. The resulting estimate is

$$\text{CV}_K = \frac{1}{K} \sum_{k=1}^K \text{MSE}_k, \quad (18)$$

where MSE_k is the prediction error on fold k . Every observation is therefore used for validation once and for training $K - 1$ times. Values between five and ten folds provide a common compromise between training-set size, computational cost, and variability of the estimate [3].

As for the bootstrap, any preprocessing must be fitted using only the training portion of each fold to avoid information leakage.

D. Gradient-based optimisation

For OLS and Ridge regression the model parameters can be obtained analytically, as described in Eqs. (6) and (10) respectively. The same optimum can instead be found iteratively by gradient descent. Starting from an initial parameter vector θ_0 , plain gradient descent updates the parameters according to

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} C(\theta_k), \quad (19)$$

where $\eta > 0$ is the learning rate [4].

For the normalized OLS cost of Eq. (5), the gradient is

$$\nabla_{\theta} C_{\text{OLS}} = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y}), \quad (20)$$

whereas for the Ridge convention of Eq. (9),

$$\nabla_{\theta} C_{\text{Ridge}} = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y}) + 2\lambda \mathbf{P}\theta, \quad (21)$$

with $\mathbf{P} = \text{diag}(0, 1, \dots, 1)$ so that the intercept remains unregularized.

Since these cost functions are quadratic, their convergence under a fixed learning rate can be related directly to the Hessian. For OLS,

$$\mathbf{H}_{\text{OLS}} = \frac{2}{n} \mathbf{X}^T \mathbf{X}, \quad (22)$$

and Ridge adds $2\lambda \mathbf{P}$. If $\lambda_{\max}$ denotes the largest Hessian eigenvalue, gradient descent is stable for

$$0 < \eta < \frac{2}{\lambda_{\max}}, \quad (23)$$

while the convergence rate also depends on the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ [4]. Thus, poor conditioning simultaneously restricts the admissible learning rate and slows convergence.

The gradients can also be evaluated by Automatic Differentiation (AD). Unlike finite differences, AD does not approximate derivatives using a finite step size; instead, exact differentiation rules are propagated through the sequence of numerical operations used to evaluate the cost function. It therefore produces derivatives accurate to floating-point precision without generating a symbolic derivative expression [4].

E. Momentum and adaptive optimisation methods

Plain gradient descent uses the current gradient together with a fixed scalar learning rate to update all

model parameters. Its convergence can be slow when the curvature of the cost function differs strongly between parameter directions. Momentum and adaptive optimisation methods modify the parameter update using information from present and previous gradients [4]. Momentum introduces memory into the update direction, whereas AdaGrad, RMSProp and Adam additionally construct parameter-dependent effective step sizes.

Momentum introduces a velocity variable,

$$\mathbf{v}_{t+1} = \gamma \mathbf{v}_t + \eta \mathbf{g}_t, \quad \theta_{t+1} = \theta_t - \mathbf{v}_{t+1}, \quad (24)$$

where $\mathbf{g}_t = \nabla C(\theta_t)$ and $0 \leq \gamma < 1$. Gradients that persist in the same direction accumulate, whereas alternating components tend to cancel, thereby damping oscillations along steep directions.

AdaGrad instead constructs a separate effective step size for each parameter by accumulating squared gradients,

$$\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t, \quad \theta_{t+1} = \theta_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{r}_t + \epsilon}}, \quad (25)$$

where ϵ is a small constant. Large-gradient directions therefore receive progressively smaller steps. Because the accumulator never decreases, however, AdaGrad can eventually make the effective learning rate very small.

RMSProp replaces this unbounded accumulation by an exponentially weighted average,

$$\mathbf{v}_t = \rho \mathbf{v}_{t-1} + (1 - \rho) \mathbf{g}_t^2, \quad \theta_{t+1} = \theta_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{v}_t + \epsilon}}, \quad (26)$$

with ρ typically 0.9 or 0.99. Adam combines this second-moment estimate with a momentum-like first moment,

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad (27)$$

with typical values $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and $\mathbf{m}_0 = \mathbf{v}_0 = 0$. However, a bias correction is necessary in this case:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \quad (28)$$

Finally

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon}}. \quad (29)$$

Thus, momentum modifies the accumulated direction of motion, whereas AdaGrad, RMSProp and Adam also introduce parameter-dependent effective step sizes [4].

F. Stochastic gradient descent

Full-batch gradient descent evaluates the gradient using all training observations before each parameter update. For an objective that can be written as an average

over the individual observations,

$$C(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n c_i(\boldsymbol{\theta}), \quad (30)$$

Stochastic Gradient Descent (SGD) instead estimates the gradient from a mini-batch $\mathcal{B}_t$ containing M observations,

$$\mathbf{g}_t = \frac{1}{M} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} c_i(\boldsymbol{\theta}_t), \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t. \quad (31)$$

The choice of gradient estimator is independent of the parameter-update rule. In this work, both full-batch and mini-batch gradients are combined with the plain, Momentum, AdaGrad, RMSProp, and Adam update rules introduced in Sec. II.E.

A complete pass through the training data is called an epoch. The limiting cases $M = 1$ and $M = n$ correspond respectively to single-sample SGD and ordinary full-batch gradient descent [4].

Using mini-batches reduces the amount of data required for each parameter update and permits several updates during one pass through the data. The price is that the mini-batch gradient is a noisy estimate of the full gradient, with fluctuations decreasing as the batch size increases. Thus, small batches can make rapid but irregular progress, whereas large batches produce smoother gradient estimates [4].

With a constant learning rate, the stochastic gradient noise generally prevents the iterates from settling exactly at the minimum. A common remedy is therefore to decrease the learning rate during training. The inverse-time schedule considered here is

$$\eta_t = \eta_0 \frac{\tau}{t + \tau}, \quad (32)$$

where η_0 is the initial learning rate and τ controls the decay. As t increases, the decreasing step size suppresses stochastic fluctuations near the optimum, although an excessively rapid decay can also slow convergence before the minimum has been reached [4].

III. METHODS AND IMPLEMENTATION

A. Data generation and preprocessing

The numerical analysis was performed using synthetic observations of Runge’s function, as explained in section II.A. A random seed of 2026 was used throughout to make the experiments reproducible. The baseline data set contained $n = 100$ observations with $\sigma = 0.1$. The data were divided into 80% training and 20% test data using `train_test_split` from Scikit-Learn [5].

For a polynomial of degree d , the design matrix $\mathbf{X}$ was constructed and the non-constant columns were standardized according to

$$X_{ij}^{\text{scaled}} = \frac{X_{ij} - \mu_j}{s_j}, \quad (33)$$

where μ_j , the mean of feature j , and s_j , the standard deviation of feature j , were computed exclusively from the training set. The same training-set statistics were subsequently applied to the test data. The intercept column was left unchanged.

B. Regression and optimisation implementations

OLS implementation and experimental setup

OLS was implemented independently using the Moore–Penrose pseudoinverse,

$$\hat{\boldsymbol{\theta}} = \mathbf{X}^+ \mathbf{y}, \quad (34)$$

computed with `numpy.linalg.pinv`. The MSE and R^2 metrics (Eqs. 7 and 8) were also implemented directly from their definitions.

For the baseline analysis, polynomial degrees $d = 1, \dots, 15$ were fitted and evaluated on the same training and test sets. In addition to the prediction scores, the fitted coefficients θ_j were recorded as the polynomial degree increased. Two further sensitivity studies were performed. First, the number of observations was varied over $n = 100, 200$, and 400 while keeping $\sigma = 0.1$. Second, the noise level was varied over $\sigma = 0.05, 0.10, 0.20$, and 0.30 while keeping $n = 200$. For the noise study, the same random seed and sample size were used in each case.

Ridge implementation and experimental setup

Ridge regression was implemented using the same design matrices, train–test split, and preprocessing procedure as for OLS, allowing the two methods to be compared on identical data. For the normalized cost function of Eq. 9, the coefficients were obtained by solving

$$(\mathbf{X}^T \mathbf{X} + n\lambda \mathbf{P}) \hat{\boldsymbol{\theta}} = \mathbf{X}^T \mathbf{y}. \quad (35)$$

The linear system was solved directly rather than by explicitly computing a matrix inverse.

For the data set $n = 200$ and $\sigma = 0.1$, polynomial degrees $d = 1, \dots, 15$ were evaluated for seven logarithmically spaced penalties, $\lambda = 10^{-6}, \dots, 10^0$. Training and test MSE and R^2 , together with the fitted coefficients, were recorded for every (d, λ) combination. Representative fitted functions were inspected for $d = 2, 5, 10$, and 15 at $\lambda = 10^{-6}, 10^{-3}$, and 1 . This range was chosen to

span the transition from a nearly unregularized model to strong regularisation [1].

To examine the regularisation mechanism directly, the singular values of the standardized non-intercept training design matrix were also computed for $d = 15$. Since the training set contains $n_{\text{tr}} = 160$ observations (80% of the data set $n = 200$), the shrinkage of each singular-value mode was evaluated as

$$s_i(\lambda) = \frac{\sigma_i^2}{\sigma_i^2 + n_{\text{tr}}\lambda}. \quad (36)$$

Gradient descent and AD

Plain gradient descent with a fixed learning rate was implemented independently for OLS and Ridge regression. A representative data set with $n = 200$, $\sigma = 0.1$, and polynomial degree $d = 5$ was divided into 80% training and 20% test data. The polynomial features were standardized using the same training-set preprocessing procedure as in the previous analyses. For Ridge, $\lambda = 10^{-2}$ was used.

The OLS and Ridge gradients were implemented analytically from the expressions in Eqs. (20) and (21), and independently by AD using `jax.grad` from JAX [6]. JAX was configured to use 64-bit floating-point arithmetic so that the two gradient implementations could be compared at machine precision. All gradient-descent runs used the same initial parameter vector and were stopped when the gradient norm fell below 10^{-8} .

The Hessian eigenvalues were computed for both regression methods. From these, the stability limit $\eta_{\text{max}} = 2/\lambda_{\text{max}}$ and the constant-step optimum $\eta_{\text{opt}} = 2/(\lambda_{\text{max}} + \lambda_{\text{min}})$ were evaluated [4]. Gradient descent was first run using η_{opt} with both the analytical and automatically differentiated gradients. A second study used $\eta/\eta_{\text{max}} = 0.10, 0.50, 0.90, 0.99$, and 1.01 to examine the dependence of convergence on the learning rate and to test the theoretical stability boundary.

Momentum and adaptive optimizer comparison

The optimisation study was extended using momentum, AdaGrad, RMSProp and Adam. The same representative regression problem as in the plain gradient-descent analysis was retained: $n = 200$, $\sigma = 0.1$, and polynomial degree $d = 5$, with $\lambda = 10^{-2}$ for Ridge. The same train-test split, feature standardization and initial parameter vector were used for every optimizer.

The analytical gradients were used for this comparison. Momentum was implemented with $\gamma = 0.9$, RMSProp with $\rho = 0.9$, and Adam with $\beta_1 = 0.9$, $\beta_2 = 0.999$; a numerical constant $\epsilon = 10^{-8}$ was used in the adaptive denominators.

Convergence was measured relative to the corresponding closed-form OLS or Ridge solution,

$$\epsilon_\theta = \frac{\|\theta_t - \hat{\theta}_{\text{closed}}\|_2}{\|\hat{\theta}_{\text{closed}}\|_2}, \quad (37)$$

with a target accuracy $\epsilon_\theta < 10^{-4}$ and a maximum of 5×10^4 iterations. Because the numerical value of the learning rate does not have the same meaning for plain and adaptive optimizers, each method was evaluated over its own range of learning rates [4]. For each optimizer, the fastest converged run within the tested learning-rate grid was retained for the convergence comparison.

Lasso implementation and validation

Lasso regression was studied using the same representative data set employed in the optimisation experiments, with $n = 200$, $\sigma = 0.1$, polynomial degree $d = 5$, and an 80/20 train-test split. The same feature-standardization procedure was applied, with the intercept left unscaled and unregularized.

The Lasso objective was implemented using the normalized convention of Sec. II.B. An analytical subgradient was constructed using $\text{sign}(\theta_j)$ for the penalized coefficients, choosing zero as the subgradient at $\theta_j = 0$. The behaviour of automatic differentiation at the non-differentiable point was also examined with JAX. For $|\theta|$, `jax.grad` returned +1 at $\theta = 0$, which is not an ordinary derivative but is a valid element of the subgradient interval $[-1, 1]$.

For an independent reference solution, cyclic coordinate descent with the soft-thresholding operator was implemented following [2]. Seven penalties, $\lambda = 10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}, 10^{-1}$, were evaluated. The implementation was validated against the Lasso estimator from Scikit-Learn [5]. Since Scikit-Learn minimizes $\|y - X\theta\|_2^2/(2n) + \alpha\|\theta\|_1$, its parameter was set to $\alpha = \lambda/2$ to match the convention used here. Agreement was additionally checked using the Lasso KKT conditions.

Finally, for $\lambda = 10^{-2}$, the subgradient formulation was optimized with plain gradient descent, momentum, AdaGrad, RMSProp, and Adam. The coordinate-descent solution was used as the reference, and convergence was defined by a relative coefficient error below 10^{-3} .

Stochastic gradient descent

The optimisation analysis was finally extended to mini-batch stochastic gradient descent using the same $n = 200$, $\sigma = 0.1$, degree-5 data set, train-test split, feature scaling, and initial parameter vector as in the previous optimisation experiments. The training set therefore contained $n_{\text{tr}} = 160$ observations. OLS and Ridge were

compared with their analytical reference solutions, while the coordinate-descent solution was used as the reference for Lasso.

At the beginning of every epoch, the training observations were randomly shuffled and divided into mini-batches. For each mini-batch, the corresponding OLS, Ridge, or Lasso gradient/subgradient was evaluated using only the observations in that batch. The resulting stochastic gradient was then combined with the same five parameter-update rules considered previously: plain gradient descent, Momentum, AdaGrad, RMSProp, and Adam.

To isolate the effect of replacing the full gradient by a stochastic mini-batch estimate, the main optimizer comparison used a fixed mini-batch size $M = 32$ and a constant learning rate. For each update rule, separate learning-rate grids were explored for the full-batch and mini-batch cases. The full-batch result was selected as in Sec. III.B, while for the stochastic runs the fastest solution reaching the prescribed coefficient tolerance was retained. If no tested learning rate reached the target within the maximum of 10^4 epochs, the run with the smallest final relative coefficient error was retained instead.

The dependence on mini-batch size and learning-rate scheduling was studied separately using plain SGD. Batch sizes $M = 1, 8, 32$, and 160 were evaluated for up to 10^4 epochs using $\eta_0 = 0.03$. Both a constant learning rate and the inverse-time schedule of Eq. (32), with $\tau = 10$, were considered. Since $M = 160$ equals the complete training-set size, this case provides the full-batch limit of the same implementation.

Convergence relative to the reference coefficients was measured using Eq. (37), with thresholds 10^{-4} for OLS and Ridge and 10^{-3} for Lasso.

Computational effort was expressed in equivalent complete passes through the training data. A full-batch update corresponds to one complete data pass, whereas the cost of a mini-batch update scales with the fraction M/n_{tr} of the training observations used in that update.

C. Bootstrap bias–variance analysis

The bias–variance behaviour was first investigated qualitatively by comparing the OLS training and test MSE as functions of polynomial complexity for $n = 100, 200$, and 400 , with fixed noise level $\sigma = 0.1$. The degree range was adapted to each sample size because the onset of the high-variance regime occurred at different model complexities. Specifically, $d = 1, \dots, 15$ was used for $n = 100$, $d = 1, \dots, 25$ for $n = 200$, and $d = 1, \dots, 40$ for $n = 400$. This allows the relevant transition from under-fitting to over-fitting to be displayed for each sample size.

The bias–variance decomposition was then estimated

using bootstrap resampling over the same degree ranges. For each sample size, an 80/20 train–test split was generated and the test set was kept fixed. From the corresponding training set, $B = 100$ bootstrap replicas were obtained by sampling observations with replacement. A separate OLS polynomial model was fitted to each replica, with feature centering and scaling recomputed within each bootstrap sample and then applied to the fixed test set. The resulting ensemble of predictions was used to estimate the expected prediction error, squared bias, and variance as functions of model complexity [3].

D. Cross-validation and comparison

Five-fold and ten-fold cross-validation were applied to the OLS models using the `KFold` implementation from Scikit-Learn [5]. For each sample size $n = 100, 200$, and 400 , cross-validation was performed using the complete available data set, without an additional train–test split. The observations were randomly divided into K folds using a fixed random seed. For $K = 5$, each model was therefore trained on $4/5$ of the data and evaluated on the remaining $1/5$, whereas for $K = 10$ it was trained on $9/10$ and evaluated on the remaining $1/10$. This procedure was repeated until every fold had served once as the held-out validation set, and the reported cross-validation MSE was obtained by averaging the MSE over all K folds. The polynomial-degree ranges were those introduced in Sec. III.C.

To compare the different model-assessment strategies, the cross-validation curves were plotted together with the ordinary OLS test MSE, the Ridge test MSE for three representative regularisation strengths, $\lambda = 10^{-6}, 10^{-3}$, and 1 , and OLS bootstrap test MSE obtained from $B = 100$ replicas.

For the final model selection, the $n = 200, \sigma = 0.1$ data set was considered using five-fold cross-validation on all available observations. Polynomial degrees $d = 1, \dots, 25$ were evaluated. OLS was selected as a function of degree only, while Ridge was evaluated for $\lambda = 10^{-6}, \dots, 1$ and Lasso for $\lambda = 10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}, 10^{-1}$. For every fold, polynomial scaling was fitted exclusively on the training portion and then applied to the validation fold. The final model for each regression method was defined as the combination of hyperparameters giving the lowest mean validation MSE across the five folds.

After model selection, the selected models were refitted using the complete $n = 200$ data set; the cross-validated MSE, rather than the refitted training error, was retained as the estimate of predictive performance.

IV. RESULTS AND DISCUSSION

A. OLS and Ridge regression

OLS results

For the baseline case, $n = 100$ and $\sigma = 0.1$, increasing the polynomial degree continuously improved the training fit: the training MSE decreased from approximately 9.6×10^{-2} at degree 1 to 1.35×10^{-2} at degree 15, while the corresponding training R^2 increased from approximately 0.02 to 0.86. This monotonic improvement is expected for OLS because each increase in polynomial degree adds an additional basis function without removing any of the previous ones.

The behaviour on the test set was markedly different, as shown in Fig. 1. The test error initially decreased as the polynomial became sufficiently flexible to reproduce the shape of Runge's function, reaching low values from approximately degree 6 onward. The minimum test MSE for this particular split was 0.018943 at degree 12, with $R^2 = 0.847$. However, degree 6 already produced an almost identical test MSE of 0.018991. Consequently, the single train-test split provides little evidence that degree 12 is intrinsically preferable to degree 6; the apparent minimum should be interpreted as a property of this particular realization rather than as a final model-selection result.

The fitted functions in Fig. 2 illustrate the transition from underfitting to increasingly high-variance behaviour. The degree-2 polynomial is unable to reproduce the narrow maximum of Runge's function. By degree 10, the central part of the function is described considerably better, although oscillatory behaviour has appeared close to the boundaries. At degree 15, the polynomial still follows the data reasonably closely through most of the sampled interval, but becomes highly unstable near the left boundary. The additional degrees of freedom make the fit increasingly sensitive to the particular noisy training sample. This behaviour is also reflected in the fitted coefficients of Fig. 3.

The influence of the number of observations is shown in Fig. 4. Increasing the data set size generally improved the test performance and reduced the instability of the high-degree models. The lowest observed test MSE decreased from 0.01894 for $n = 100$ to 0.01024 for $n = 200$ and 0.01014 for $n = 400$. The corresponding minima occurred at degrees 12, 14, and 14, respectively. This suggests that a larger training set can support a more flexible polynomial model because the additional parameters are constrained by more observations.

Increasing the noise amplitude caused the expected deterioration in predictive performance, as shown in Fig. 5. The lowest test MSE increased from 0.003108 for $\sigma = 0.05$ to 0.010240, 0.038578, and 0.085679 for $\sigma = 0.10$, 0.20, and 0.30, respectively. At the same time, the poly-

nomial degree giving the lowest test error shifted from degree 14 for $\sigma = 0.05$ to degree 13 for $\sigma = 0.30$. The corresponding maximum test R^2 decreased from approximately 0.95 to 0.40.

This behaviour is consistent with the bias-variance interpretation of polynomial regression. When the observations contain little noise, additional polynomial terms can capture more of the structure of Runge's function. As the noise level increases, however, highly flexible models can increasingly adapt to random fluctuations in the training sample, so a lower-complexity model generalizes more reliably.

Ridge results

Figure 6 compares the test MSE and R^2 obtained with OLS and with the different Ridge penalties. For small values of λ , the Ridge curves remain close to the OLS solution. It can be noticed that the small OLS error fluctuations from degree 11 forward are slightly suppressed by the Ridge curves. The lowest test MSE among the Ridge configurations considered was 0.011309, obtained for $\lambda = 10^{-6}$ and $d = 14$, with $R^2 = 0.853304$. For comparison, the minimum OLS test MSE for the same split was 0.010240, also at $d = 14$.

For $n = 200$, the weakest regularisation considered, $\lambda = 10^{-6}$, gives the closest overall approximation to Runge's function among the three representative cases (see Figure 7). The intermediate value $\lambda = 10^{-3}$ produces a smoother fit with some loss of flexibility, while $\lambda = 1$ clearly under-fits the narrow central peak. This behaviour indicates that, with a larger training set, only mild regularisation is needed to stabilize the polynomial fit. However, the value of λ that performs best depends on the sample size and on the particular train-test realization, so the weakest penalty is not necessarily optimal for all values of n .

In Figure 8, the effect of Ridge regularisation on the degree-15 coefficients is clearly visible. The unregularized OLS solution contains coefficients with magnitudes of several hundred, whereas even the weakest Ridge penalty reduces them substantially. As λ increases, the coefficients are progressively drawn toward zero.

Figure 9 shows the same regularisation effect in the singular-value basis. For each singular-value mode, Ridge multiplies its contribution to the fitted values by the shrinkage factor $s_i = \sigma_i^2 / (\sigma_i^2 + n_{\text{tr}} \lambda)$ [2]. Modes associated with large singular values have $s_i \approx 1$ and are therefore retained, whereas modes associated with small singular values have $s_i \approx 0$ and are strongly suppressed. Increasing λ shifts this transition toward larger singular values, so progressively more modes are damped. The figure therefore shows directly how Ridge reduces the contribution of the poorly constrained, high-variance directions of the degree-15 polynomial design matrix.

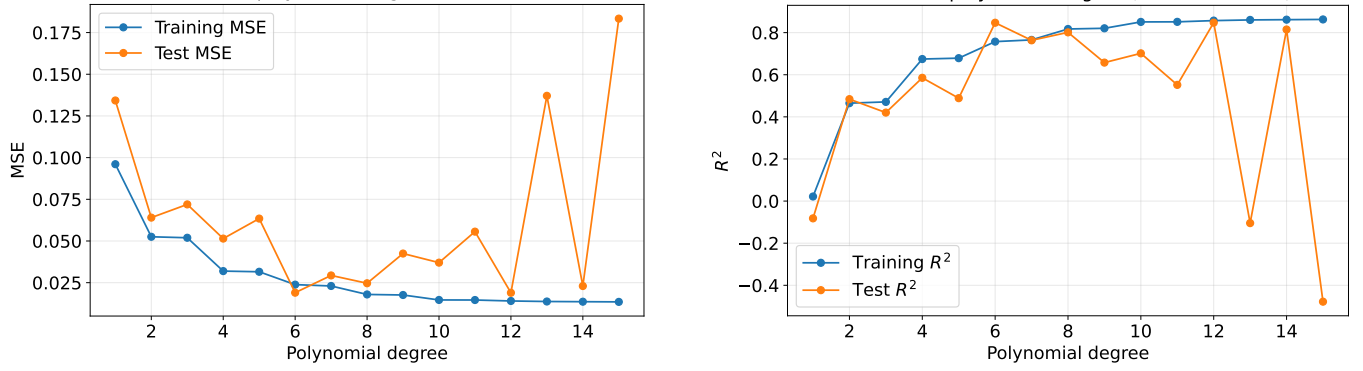


FIG. 1 Training and test performance of the OLS polynomial models for the baseline data set ($n = 100$, $\sigma = 0.1$). MSE vs polynomial degree on the left. R^2 vs polynomial degree on the right.

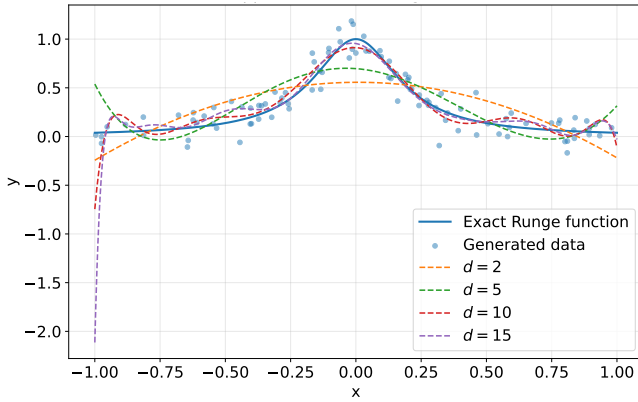


FIG. 2 Representative OLS fits for polynomial degrees 2, 5, 10, and 15 for the baseline data set ($n = 100$, $\sigma = 0.1$).

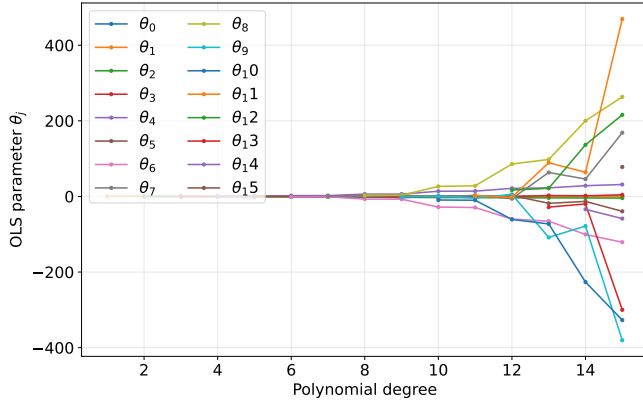


FIG. 3 OLS coefficients as a function of polynomial degree for the baseline data set ($n = 100$, $\sigma = 0.1$).

B. Bias-variance analysis and cross-validation

Bias-variance analysis

The bias-variance behaviour can first be identified qualitatively from the training and test errors in Fig. 10.

At low polynomial degree, both errors are relatively large because the model is too restrictive to reproduce the shape of Runge's function. Increasing the degree initially reduces both quantities as the approximation bias decreases. The training MSE continues to decrease as further polynomial terms are added, whereas the test error eventually stops improving and begins to increase or fluctuate. The resulting separation between training and test performance indicates the transition toward a high-variance regime [3].

The complexity at which this deterioration appears shifts substantially with sample size. For $n = 100$, the test error is already highly irregular at the largest degrees considered, despite the continued decrease of the training error. For $n = 200$, the test MSE remains close to its minimum over a broader range but rises sharply above approximately $d = 20$. In contrast, for $n = 400$ the training and test curves remain close to one another over much of the interval, and the test error increases only gradually at the highest degrees. These results indicate that a larger training sample can support a more flexible polynomial model before sensitivity to the training data becomes dominant. The behaviour of test and training MSE shown in Fig 10 is consistent with Fig. 2.11 of Hastie, Tibshirani and Friedman [7].

The bootstrap decomposition in Fig. 11 makes the origin of this behaviour explicit. At low polynomial degree, the prediction error is mainly associated with the bias term. As the polynomial degree increases, the bias initially decreases because the model becomes sufficiently flexible to approximate the underlying function. The variance, however, increases because the estimated polynomial becomes progressively more sensitive to which observations appear in each bootstrap training sample. Increasing n shifts the bias-variance trade-off toward more complex models.

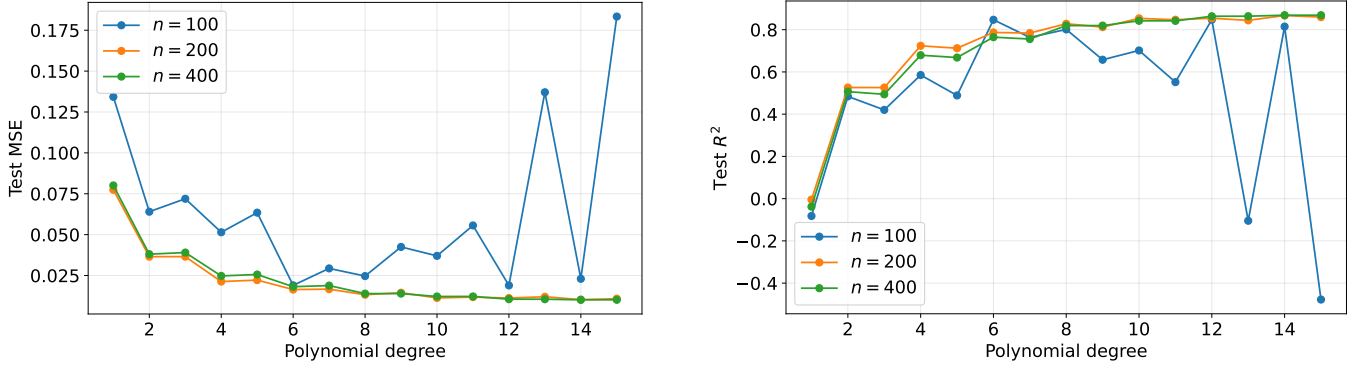


FIG. 4 Effect of sample size on the test performance of OLS polynomial regression for fixed noise level $\sigma = 0.1$. Test MSE vs polynomial degree on the left. Test R^2 vs polynomial degree on the right.

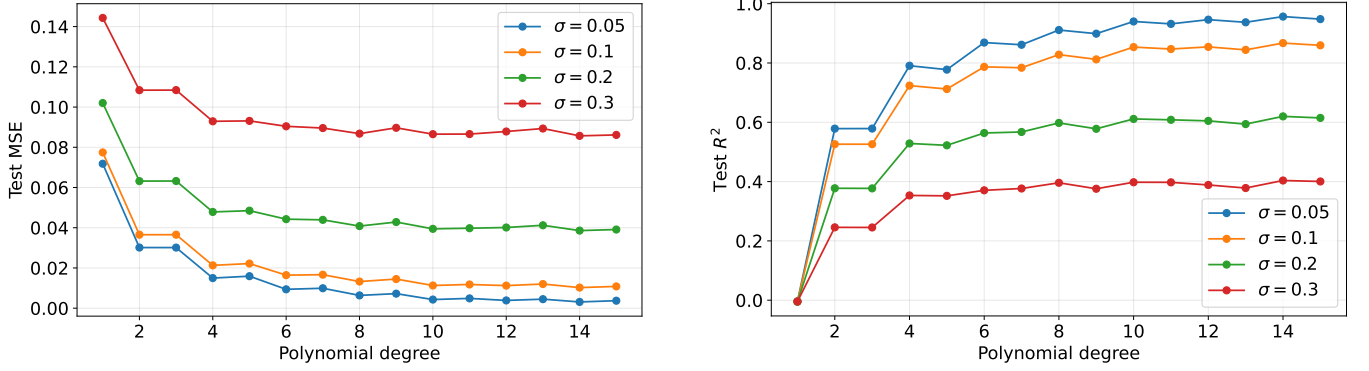


FIG. 5 Effect of the Gaussian noise level on OLS polynomial regression for $n = 200$. Test MSE vs polynomial degree on the left. Test R^2 vs polynomial degree on the right.

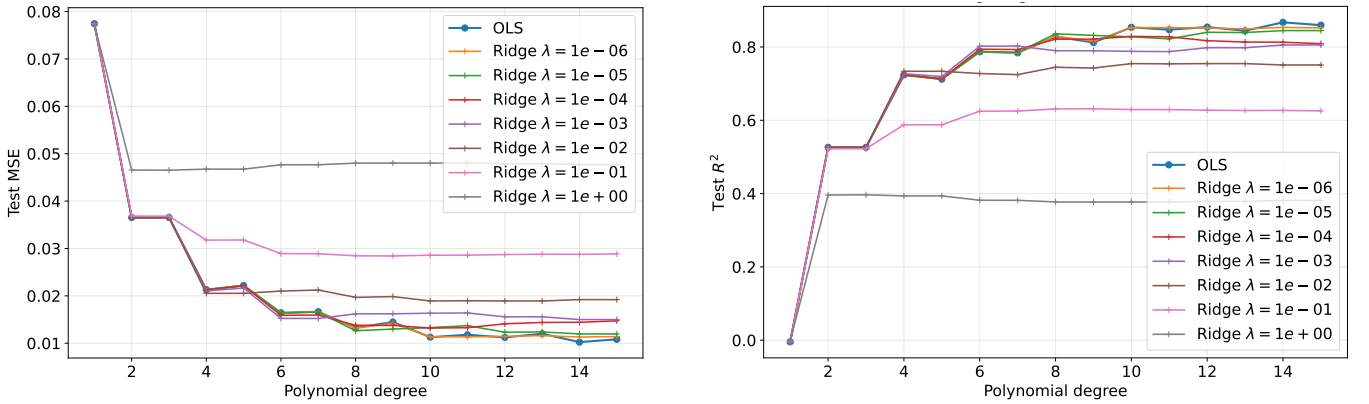


FIG. 6 Comparison of OLS and Ridge test performance for $n = 200$ and $\sigma = 0.1$. Test MSE vs polynomial degree on the left. Test R^2 vs polynomial degree on the right.

Cross-validation and comparison of model assessment

Figure 12 compares the ordinary OLS test MSE, Ridge regression, bootstrap resampling, and five- and ten-fold cross-validation. At low and intermediate polynomial degrees, all methods give similar errors and identify the same broad region of useful model complexity. The five- and ten-fold curves also agree closely with each other.

Their minima occur at $d = 12, 14$, and 16 for $n = 100, 200$, and 400 , respectively, with very similar MSE values for the two choices of K . This indicates that the precise choice between five and ten folds has little effect on the estimated optimal complexity in this experiment.

For $n = 100$ and $n = 200$, the cross-validation curves broadly follow the ordinary OLS test error (red line), including its deterioration at high polynomial degree. For

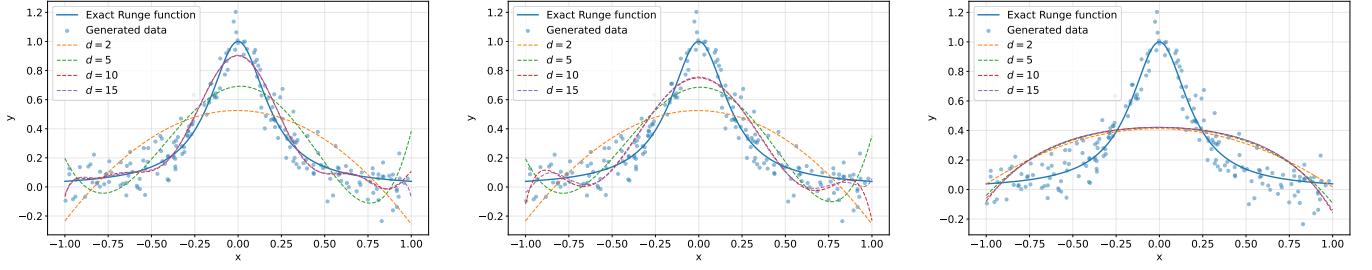


FIG. 7 Representative Ridge polynomial fits for weak ($\lambda = 10^{-6}$), intermediate ($\lambda = 10^{-3}$), and strong ($\lambda = 1$) regularisation for $n = 200$ and $\sigma = 0.1$.

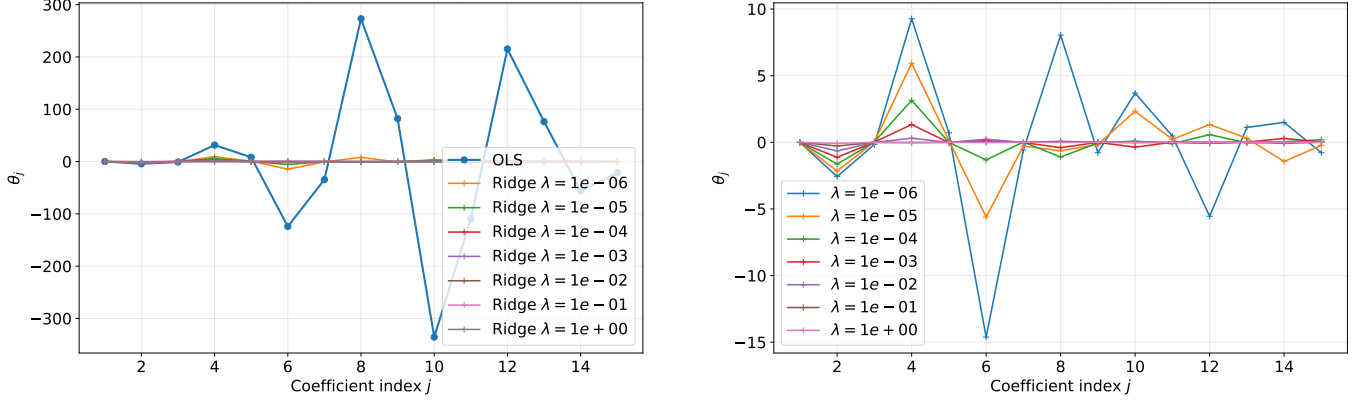


FIG. 8 Comparison of the non-intercept coefficients of the degree-15 OLS and Ridge models on the left for $n = 200$ and $\sigma = 0.1$. Zoom in on the coefficients of the degree-15 Ridge models on the right.

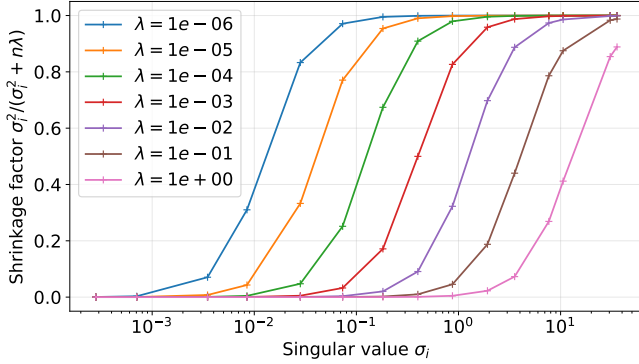


FIG. 9 Ridge shrinkage factor $\sigma_i^2 / (\sigma_i^2 + n_{\text{tr}}\lambda)$ as a function of the singular values of the standardized degree-15 design matrix for $n = 200$ and $\sigma = 0.1$.

$n = 400$, however, the fixed-split OLS error remains comparatively low at high degree while the cross-validation error increases. This difference is expected because the ordinary OLS curve depends on one particular train-test split, whereas cross-validation averages over several different held-out subsets and therefore exposes high-degree instability that may not appear in a favourable single split.

The bootstrap estimate is generally the most pessimistic and becomes unstable at lower polynomial de-

gree, particularly for the smaller data sets. This does not mean that the bootstrap error should be regarded as the “true” or absolute prediction error simply because it is the largest. Bootstrap and cross-validation estimate generalisation behaviour through different resampling procedures. A bootstrap sample contains only about 63.2% of the original observations as distinct points, which is especially restrictive for high-order polynomial regression and increases the sensitivity of the fit to the particular resample [3]. Consequently, bootstrap tends to favour lower model complexity than cross-validation for $n = 100$ and $n = 200$. For $n = 400$, the low-error regions of the two methods overlap much more closely, showing that their conclusions become more consistent as the available data increase.

Ridge behaves differently: increasing λ progressively flattens the MSE curve because the penalty suppresses the poorly constrained high-order polynomial components. Weak regularisation remains close to OLS at moderate degrees while preventing its strongest high-degree fluctuations, whereas $\lambda = 1$ produces an almost degree-independent but substantially larger error, indicating excessive regularisation and underfitting.

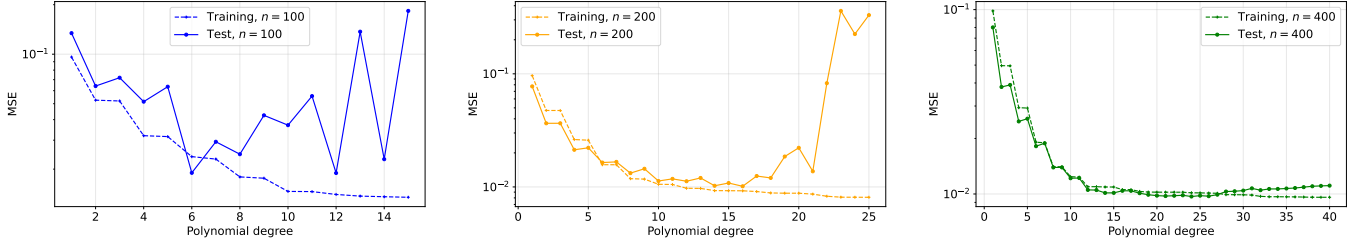


FIG. 10 Training and test MSE for OLS as functions of polynomial complexity for $n = 100, 200$, and 400 , with $\sigma = 0.1$.

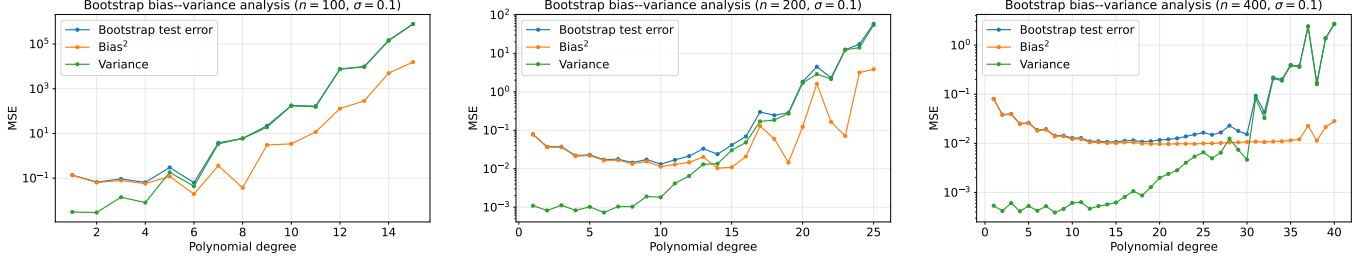


FIG. 11 Test MSE for OLS, squared bias, and variance for $n = 100, 200$, and 400 , with $\sigma = 0.1$ using bootstrap resampling.

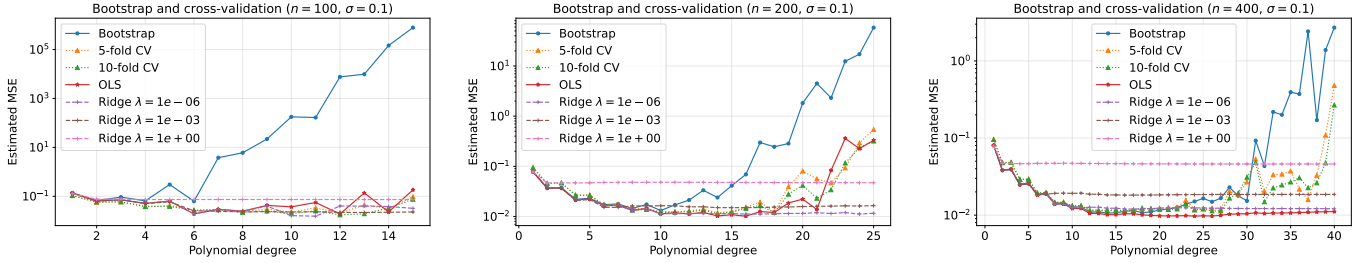


FIG. 12 Comparison of model-assessment and regularisation strategies for $n = 100, 200$, and 400 , with $\sigma = 0.1$. Shown are the ordinary OLS test MSE from the fixed train-test split, OLS bootstrap test MSE, five- and ten-fold OLS cross-test MSE, and Ridge test MSE for $\lambda = 10^{-6}, 10^{-3}$, and 1 .

C. Gradient descent, adaptive methods, Lasso and SGD

Plain gradient descent and AD

The analytical and automatically differentiated gradients agree to machine precision for both regression methods, and both lead to the same gradient descent solution (see Table I). The fitted parameters reproduce the corresponding closed-form solutions to approximately 10^{-9} . Ridge converges in substantially fewer iterations than OLS, consistent with the reduction of the Hessian condition number from approximately 5.16×10^2 to 1.82×10^2 . regularisation therefore improves not only statistical stability, but also the conditioning of the optimisation problem [4].

The direct comparison with the regression implementations used in Sec. III.B provides an additional consistency check. For the same $n = 200$, $\sigma = 0.1$, $d = 5$ data set, and $\lambda = 10^{-2}$ for Ridge, gradient descent reproduces the earlier analytical test errors to approximately 10^{-11} in

TABLE I Gradient verification and convergence of plain gradient descent for the representative case $n = 200$, $\sigma = 0.1$, and $d = 5$. Ridge uses $\lambda = 10^{-2}$. The final rows compare the gradient-descent solutions directly with the analytical OLS and Ridge implementations used in Sec. III.B.

Quantity	OLS	Ridge
$\max \nabla C_{AD} - \nabla C_{an} $	8.88×10^{-16}	8.88×10^{-16}
$\kappa(H)$	5.16×10^2	1.82×10^2
η_{opt}	0.356240	0.353720
η_{max}	0.356931	0.355661
GD iterations	5117	1809
$\ \theta_{GD} - \theta_{closed}\ _2$	3.78×10^{-9}	3.38×10^{-9}
Closed-form test MSE	0.022185970	0.020533871
GD test MSE	0.022185970	0.020533871
$ \Delta \text{MSE} $	2.80×10^{-11}	1.43×10^{-11}

MSE (see last three rows of Table I). The corresponding coefficient vectors agree to approximately 10^{-9} . Thus,

gradient descent changes the optimisation procedure but not the fitted regression solution.

The effect of the fixed learning rate is shown in Fig. 13. Small values of η produce stable but slow convergence, while increasing the learning rate toward the theoretical limit substantially reduces the number of required iterations. For both OLS and Ridge, all tested values up to $0.99\eta_{\max}$ converge, whereas $1.01\eta_{\max}$ diverges. The observed transition therefore agrees closely with the Hessian stability condition $\eta < 2/\lambda_{\max}(H)$ [4]. Ridge converges more rapidly throughout the stable region, consistent with its lower Hessian condition number.

Momentum and adaptive optimizers

Figure 14 separates two independent choices in the optimisation procedure: the gradient can be evaluated from the complete training set or estimated from a mini-batch, while the resulting gradient can be used with any of the five update rules. Solid and dashed curves of the same colour therefore differ only in whether the gradient is full-batch or stochastic.

For the full-batch calculations, Momentum remains the fastest of the tested methods for both OLS and Ridge. Introducing mini-batch gradients generally increases the number of equivalent data passes required to approach the same minimum. This is particularly clear for OLS: none of the mini-batch methods reached the strict coefficient criterion $\epsilon_{\theta} < 10^{-4}$ within 10^4 epochs, although Adam obtained the smallest final coefficient error, $\epsilon_{\theta} = 1.24 \times 10^{-3}$. For Ridge, stochastic Momentum did reach the prescribed tolerance, requiring 6310 data passes compared with only 185 for full-batch Momentum.

The stochastic curves also display the characteristic fluctuations expected from mini-batch gradients. Their cost can approach the deterministic minimum closely, but the residual gradient noise prevents the trajectories from following the smooth full-batch convergence curves. For this small training set of only 160 observations, the stochastic calculation therefore provides no computational advantage when cost is measured in equivalent data passes. Its principal advantage would be expected for substantially larger data sets, where evaluating the full gradient before every update becomes expensive [4].

The sensitivity study in Fig. 15 also demonstrates that the numerical value of η cannot be compared directly between optimizers. Plain gradient descent operates efficiently for learning rates of order 10^{-1} , whereas the successful RMSProp values in the present implementation are of order 10^{-5} . Momentum, AdaGrad and Adam improve strongly as the learning rate is increased over much of the tested range. Consequently, comparing all methods at a single common value of η would give a misleading assessment of their relative performance [4]. The values

reported in Table II should be interpreted as the best learning rates among those tested rather than as rigorously determined global optima.

Lasso regression

Figure 16 illustrates the characteristic sparsity induced by the L_1 penalty. For the weakest penalties, all five non-intercept polynomial coefficients remain non-zero. As λ increases, coefficients are successively driven exactly to zero. This differs qualitatively from Ridge, which continuously shrinks coefficients but does not normally force them exactly to zero [2].

The distinction between the three regression methods is shown directly in Fig. 17. OLS gives the largest unconstrained coefficients, while Ridge reduces their magnitudes continuously. Lasso produces a sparse solution: at $\lambda = 10^{-2}$ only two of the five non-intercept coefficients remain non-zero. Despite this much simpler parametrization, its test MSE is 0.020672, close to the Ridge value 0.020534 obtained for the same $d = 5$ and $\lambda = 10^{-2}$ configuration, and lower than the corresponding OLS value of approximately 0.02219. These values refer only to this particular train–test realization and should not be interpreted as a general ranking of the three regression methods.

The independently implemented coordinate-descent solution agreed closely with Scikit-Learn over the complete penalty range. The Euclidean difference between the coefficient vectors was between approximately 10^{-8} and machine precision, while the maximum KKT-condition violation was of order 10^{-10} or smaller. This agreement verifies both the implementation and the conversion between the two penalty conventions. The lowest test MSE among the Lasso penalties examined was 0.020672 at $\lambda = 10^{-2}$; stronger regularisation subsequently increased the error as the model became too restricted.

Figure 18 extends the same full-batch versus mini-batch comparison to the non-smooth Lasso objective. As for OLS and Ridge, solid and dashed curves of the same colour use the same parameter-update rule and differ only in whether the gradient is evaluated from the full training set or from mini-batches of size $M = 32$.

Momentum is again the fastest configuration in both settings. Full-batch Momentum reaches the Lasso coefficient criterion in 178 complete data passes, whereas stochastic Momentum requires 265 epochs. The other stochastic methods also reach the target, but require more data passes: Adam converges in 462, AdaGrad in 1000, plain SGD in 1767, and RMSProp in 4473 epochs. Despite the noisier trajectories, their final test errors remain close to the coordinate-descent reference value 0.020672. Thus, for this Lasso problem mini-batch optimisation can recover essentially the same fitted model,

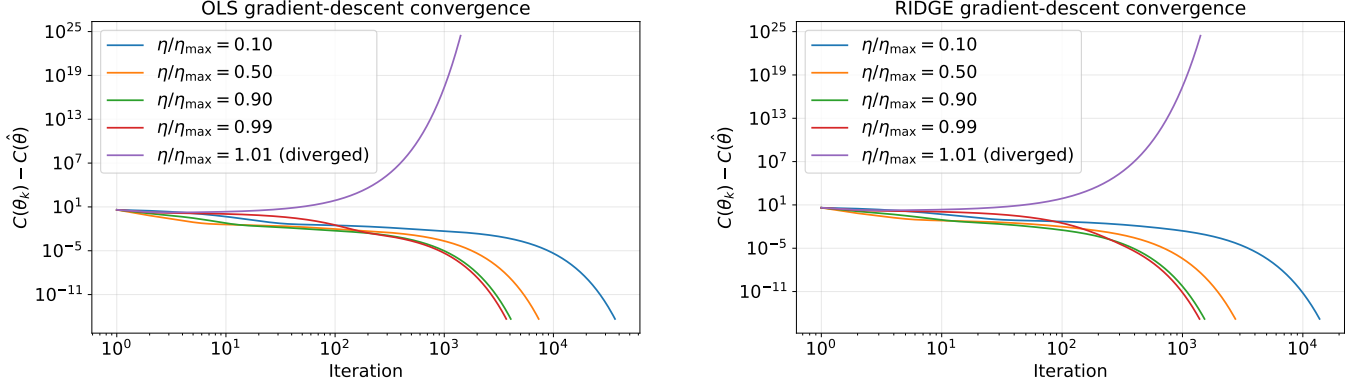


FIG. 13 Convergence of plain gradient descent for OLS (left) and Ridge regression (right) for different learning rates expressed relative to the theoretical stability limit $\eta_{\max} = 2/\lambda_{\max}(H)$. The vertical axis shows the excess cost relative to the corresponding closed-form minimum.

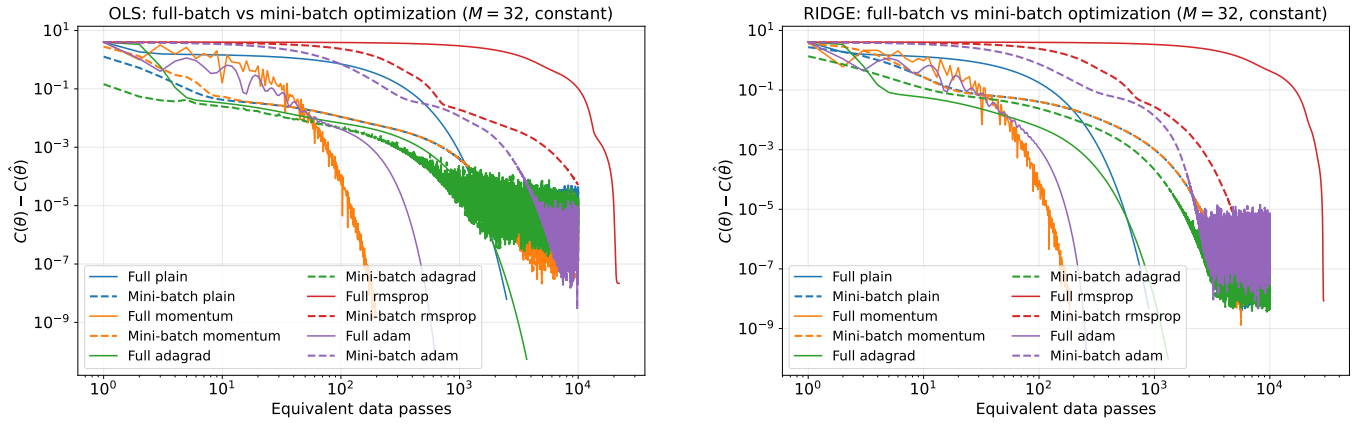


FIG. 14 Comparison of full-batch and mini-batch optimisation for OLS (left) and Ridge regression (right). Solid curves use gradients evaluated from the complete training set, whereas dashed curves use mini-batch gradients with $M = 32$. The horizontal axis is expressed in equivalent complete passes through the training data.

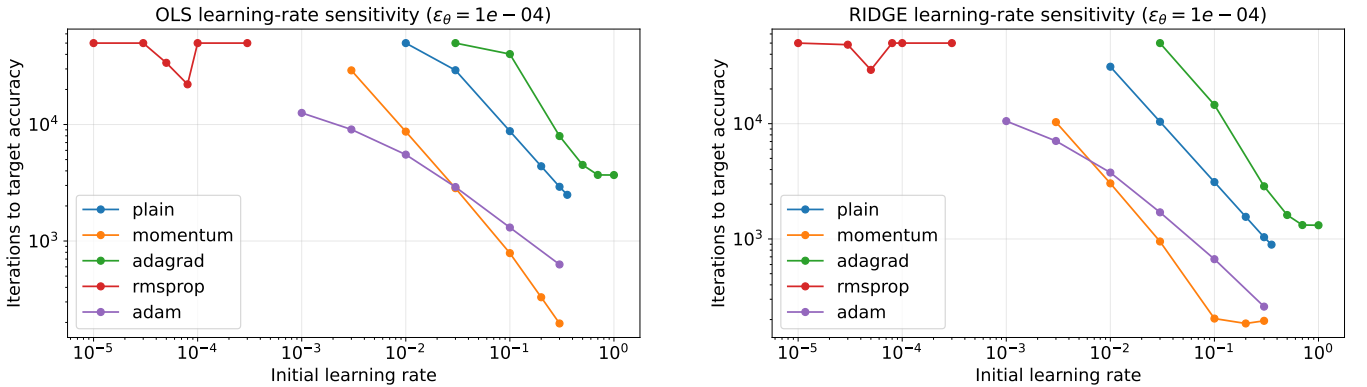


FIG. 15 Sensitivity of the five optimisation methods to the initial learning rate η for OLS (left) and Ridge (right). The vertical axis gives the number of iterations required to reach target accuracy $\epsilon_{\theta} < 10^{-4}$ (see Eq. (37)).

although it does not improve computational efficiency relative to the best full-batch optimizer.

Stochastic gradient descent

The preceding optimizer comparisons established how the five update rules behave when the full gradient is

TABLE II Best tested learning rate and number of iterations required to reach $\epsilon_\theta < 10^{-4}$ (from Eq. (37)) for the OLS and Ridge optimisation problems. All converged methods reproduce essentially the same test MSE because they minimize the same regression cost function.

Optimizer	η (OLS)	Iterations (OLS)	η (Ridge)	Iterations (Ridge)
Plain GD	3.562×10^{-1}	2492	3.537×10^{-1}	893
Momentum	3.0×10^{-1}	196	2.0×10^{-1}	185
AdaGrad	1.0	3685	1.0	1316
RMSProp	8.0×10^{-5}	22129	5.0×10^{-5}	29291
Adam	3.0×10^{-1}	629	3.0×10^{-1}	259

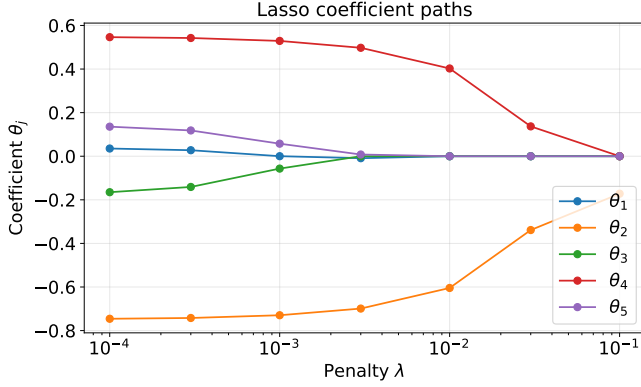


FIG. 16 Lasso coefficient paths for the degree-5 polynomial model with $n = 200$ and $\sigma = 0.1$.

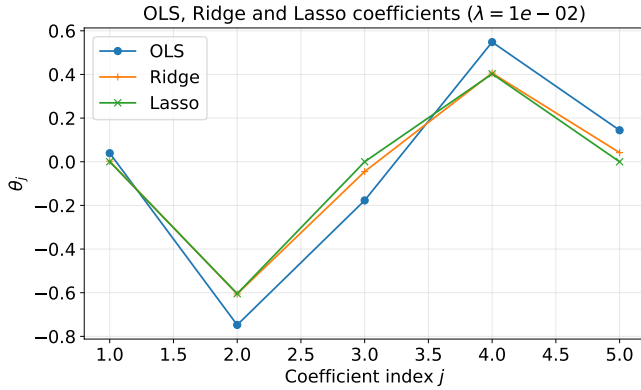


FIG. 17 Comparison of the non-intercept OLS, Ridge, and Lasso coefficients for $d = 5$ and $\lambda = 10^{-2}$.

replaced by a mini-batch estimate. The remaining SGD-specific questions are the effects of mini-batch size and of the learning-rate schedule. These are examined independently below using the plain stochastic update.

The effect of mini-batch size is shown in Fig. 19. With a constant learning rate, decreasing the batch size increases the stochastic fluctuations of the optimisation. The $M = 1$ trajectories are particularly noisy because every update is based on a single observation. Increasing M reduces the variance of the gradient estimate and produces smoother convergence. However, the largest

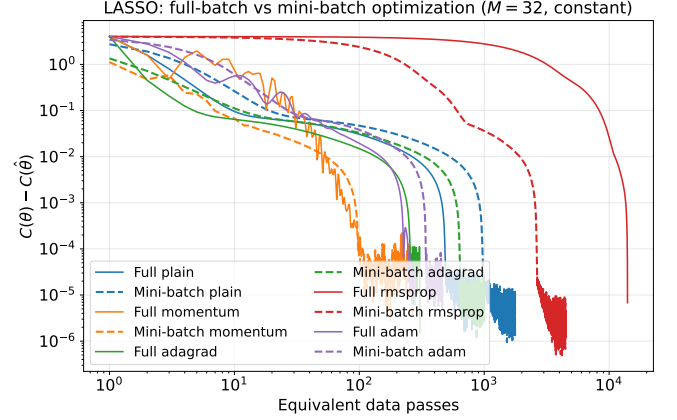


FIG. 18 Full-batch and mini-batch optimisation of the Lasso objective for $\lambda = 10^{-2}$. Solid curves use the complete training gradient and dashed curves use mini-batch gradients with $M = 32$.

batch is not necessarily the most efficient when progress is measured per epoch: $M = 160$ corresponds to full-batch gradient descent and therefore performs only one parameter update per complete pass through the training data, whereas smaller batches perform several updates during the same epoch. For the present problem, $M = 8$ and $M = 32$ provide the clearest compromise between several updates per data pass and a reduction of stochastic-gradient noise.

The inverse-time schedule in Fig. 20 produces much smoother trajectories than the constant-rate calculation. However, all three objectives level off well above their deterministic minima. In the present implementation the learning rate is reduced after every mini-batch update according to Eq. (32) with $\tau = 10$. Smaller mini-batches therefore experience more learning-rate updates during a given epoch, while for all batch sizes the step eventually becomes very small. The schedule suppresses the stochastic fluctuations effectively, but here it decays too rapidly to continue resolving the slowly converging parameter directions.

Table III summarizes the comparison in terms of the coefficient criterion and equivalent data passes. For OLS, none of the tested mini-batch configurations reached $\epsilon_\theta < 10^{-4}$ within 10^4 epochs. Adam gave the smallest final

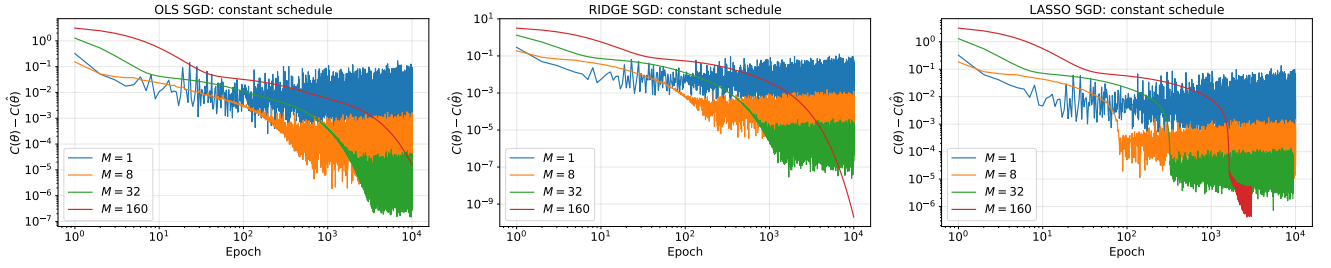


FIG. 19 Effect of mini-batch size on plain SGD with a constant learning rate $\eta = 0.03$ for OLS (left), Ridge (centre), and Lasso (right). The training set contains 160 observations, so $M = 160$ corresponds to the full-batch limit.

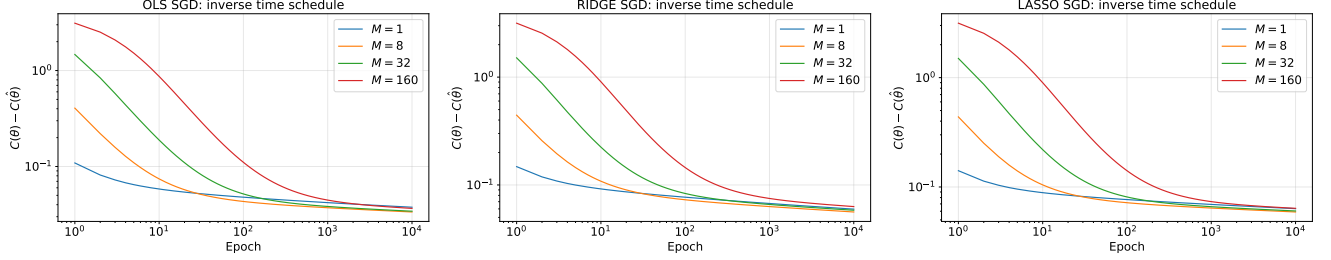


FIG. 20 Effect of mini-batch size on plain SGD using the inverse-time learning-rate schedule for OLS (left), Ridge (centre), and Lasso (right). The same initial learning rate $\eta_0 = 0.03$ is used for all batch sizes.

coefficient error, $\epsilon_\theta = 1.24 \times 10^{-3}$, while still producing a test MSE of 0.022226, very close to the analytical OLS value 0.022186.

D. Final model comparison

Table IV summarizes the final model selection. Among the tested configurations, OLS gives the lowest five-fold cross-validated error.

The two-dimensional dependence on polynomial degree and regularisation strength is shown in Fig. 21. Strong penalties produce systematically larger errors because they over-shrink the polynomial coefficients and prevent the model from reproducing the narrow structure of Runge’s function. As the penalty is weakened, both methods approach the low-error region obtained by the unregularized model. In fact, the selected penalty is the smallest value examined for both Ridge and Lasso. This indicates that, for $n = 200$ and $\sigma = 0.1$, cross-validation provides little evidence that substantial regularisation is beneficial.

Because both regularized minima occur at the lower boundary of the tested λ grids, the precise optimum penalty is not bracketed by the present search. Smaller positive values of λ would make Ridge and Lasso approach the OLS solution still more closely.

Figure 22 shows that the three methods behave similarly over the intermediate-complexity region, but differ strongly once the polynomial degree becomes large. For this particular Runge-function problem, OLS has the

advantage of simplicity and achieves the lowest cross-validated error when the polynomial degree itself is selected appropriately. Its main weakness is sensitivity to model complexity: once unnecessary high-order terms are introduced, the ill-conditioned polynomial design matrix leads to rapidly increasing variance and unstable predictions.

Ridge and Lasso do not attain quite as low a minimum, consistent with the bias introduced by regularisation, but their errors remain far more stable as the nominal polynomial degree is increased. Thus the regularized methods trade a small increase in bias near the optimum for a substantial reduction of high-complexity variance.

TABLE III Best full-batch and mini-batch optimisation results for the representative $d = 5$ problems. Mini-batch results use $M = 32$ and a constant learning rate. Data passes denote complete equivalents of processing the training set. An asterisk indicates that the prescribed coefficient tolerance was not reached within 10^4 epochs; the smallest final coefficient error among the tested stochastic runs is reported instead.

	Model	Full-batch rule	Passes	Mini-batch rule	Passes	ϵ_θ	Test MSE
	OLS	Momentum	196	Adam	10 000*	1.24×10^{-3}	0.022226
	Ridge	Momentum	185	Momentum	6310	8.81×10^{-5}	0.020537
	Lasso	Momentum	178	Momentum	265	9.40×10^{-4}	0.020641

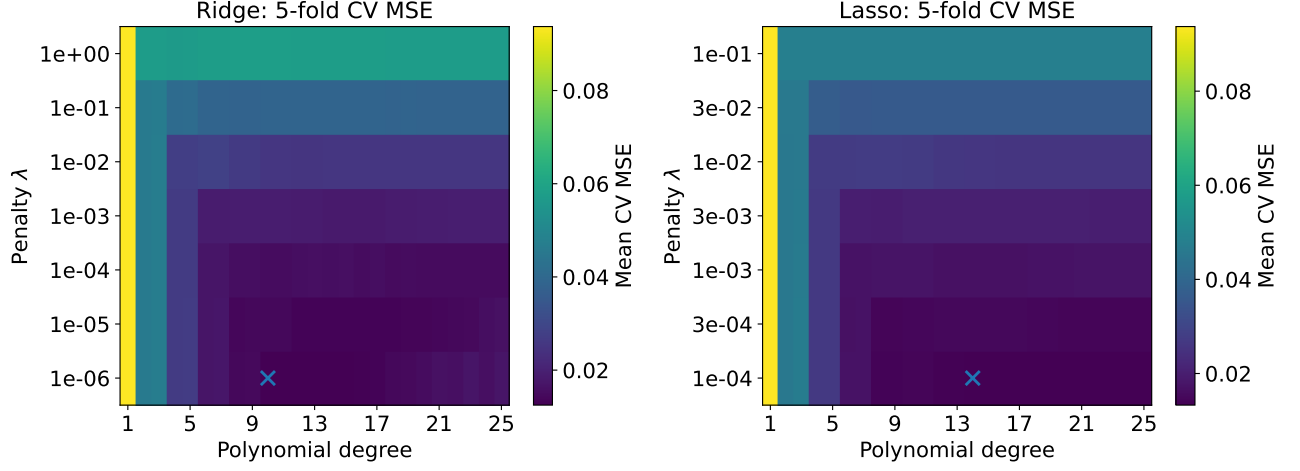


FIG. 21 Five-fold cross-validated MSE for Ridge (left) and Lasso (right) as functions of polynomial degree and regularisation strength. The cross marks the minimum within each investigated hyper-parameter grid.

TABLE IV Final five-fold cross-validation model selection for $n = 200$ and $\sigma = 0.1$.

Method	Degree d	λ	CV MSE
OLS	14	—	0.011852
Ridge	10	10^{-6}	0.012489
Lasso	14	10^{-4}	0.013282

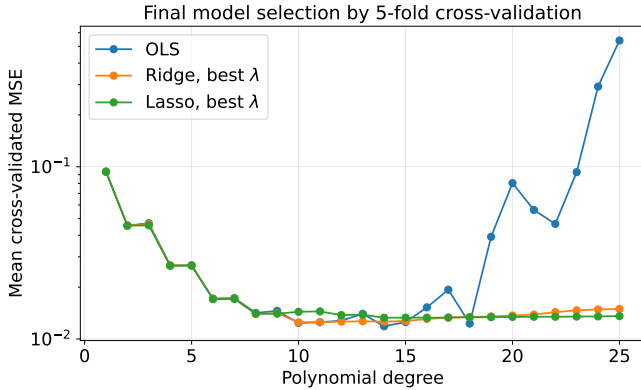


FIG. 22 Final five-fold cross-validation comparison of OLS, Ridge, and Lasso as functions of polynomial degree. For Ridge and Lasso, the curve at each degree shows the lowest CV MSE over the tested values of λ .

V. CONCLUSIONS AND PERSPECTIVES

This work used polynomial regression of noisy Runge-function data to examine the relation between model complexity, generalisation, regularisation, resampling, and numerical optimisation. The OLS analysis demonstrated the usual transition from underfitting to high-variance behaviour: increasing the polynomial degree initially reduced prediction error, but sufficiently flexible models became highly sensitive to the particular training sample. Increasing the number of observations delayed this transition, while stronger observation noise reduced the useful model complexity. The bootstrap decomposition confirmed that the deterioration of high-degree models is associated with increasing variance, while cross-validation provided a more reliable model selection criterion than a single train-test split.

Ridge and Lasso reduced this sensitivity by introducing bias in exchange for lower variance. Ridge suppressed poorly constrained singular-value directions, whereas Lasso additionally produced sparse coefficient vectors. Nevertheless, for the final $n = 200$, $\sigma = 0.1$ five-fold cross-validation study, OLS gave the lowest mean error: 0.011852 at degree 14, compared with 0.012489 for Ridge and 0.013282 for Lasso. Both regularised optima occurred at the smallest tested penalties, indicating that only weak regularisation was favoured for this data set. The regularised methods were, however, considerably more robust when unnecessarily high polynomial degrees were considered.

The optimisation experiments provided independent verification of the regression implementations. Analytical and automatically differentiated gradients agreed to floating-point precision, and iterative gradient descent reproduced the closed-form OLS and Ridge solutions. Momentum was the fastest of the tested full-batch update rules for the representative problems. Using mini-batch gradients introduced the expected stochastic fluctuations and did not reduce the computational cost for the small training set considered here, although this conclusion should not be extrapolated directly to large-scale problems where full-gradient evaluations can be substantially more expensive.

Several improvements would be natural extensions of this work. The Ridge and Lasso penalty grids could be extended toward smaller values because their cross-validated minima occurred at the lower boundaries of the investigated ranges. Repeating the complete analysis over several independently generated data sets or using repeated cross-validation would provide uncertainty estimates for the selected hyperparameters. Finally, replacing the monomial polynomial basis by an orthogonal basis, such as Chebyshev polynomials, would be useful for separating statistical over-fitting from numerical ill-conditioning and would provide a natural extension of the Runge-function study.

Appendix A: Use of AI/LLM Tools

1. Tools Used

ChatGPT (OpenAI, accessed through the ChatGPT web interface, September–October 2026) was used during the development of the project. No other large-language-model tool was used.

2. Writing and Editing

ChatGPT was used both as a conceptual tutor and as a writing assistant. The contribution to each part of the report is summarized below according to the contribution levels defined in the FYS-STK3155 Applied Data Analytics and Machine Learning course guidelines.

For the Level-3 contributions, prompts generally supplied the relevant project exercise, lecture-note section, current source code, numerical output, and/or generated figures and asked ChatGPT to explain the underlying concept or draft text consistent with the existing report. The generated text was used as a starting point and was iteratively revised to match the notation, numerical results, and structure of the final report. Only material understood and accepted by the author was retained.

3. Programming Assistance

ChatGPT was also used during code development. Its role was to suggest implementations, reorganizations, debugging strategies, and plotting routines. The main contributions are summarized below.

All LLM-assisted numerical results used in the report were generated by running the final source code. The LLM was not used to fabricate numerical data. Suggested implementations were tested and compared, where applicable, with closed-form OLS and Ridge solutions, automatic-differentiation results, KKT conditions, and the scikit-learn Lasso implementation.

TABLE V LLM contributions to the written report. Level 3 denotes substantial generative assistance followed by revision and verification by the author.

Section	Level	Role of ChatGPT
Abstract	3	Drafted from the completed report and final numerical results; numerical values and interpretation were checked against the reported results.
Introduction	3	Used the project description and report-writing guidelines to draft and revise the motivation, aims, and report structure.
Theory	2	Assisted with explanations and drafts of OLS, Ridge, Lasso, bias–variance decomposition, gradient descent, adaptive optimizers, and stochastic gradient descent. The supplied lecture notes were specified as the primary sources; notation and equations were checked and adapted to the conventions used in the report.
Methods and Implementation	2	Helped convert the implemented numerical workflow into concise methodological descriptions and organize the experimental settings. Parameter values and algorithmic details were checked against the source code.
Results and Discussion	2	Assisted in interpreting the generated figures and terminal outputs and in drafting and restructuring discussion paragraphs. Reported numerical values were taken from the author’s program output and figures rather than generated by the LLM.
Conclusions and Perspectives	2	Drafted a synthesis of the main findings, limitations, and possible future extensions based on the completed analysis.
AI/LLM Declaration	3	Drafted from the course declaration guidelines and the record of LLM use during the project.

TABLE VI LLM assistance in the submitted source files.

File	Level	Role of ChatGPT
<code>main.py</code>	2	Suggested the organization of several project-part workflows and provided partial implementations for later analyses, including optimization, Lasso, SGD, and final cross-validation model selection. The code was integrated, modified, and executed by the author.
<code>utilities.py</code>	2	Provided or assisted with several numerical routines for regression, resampling, gradient-based optimization, coordinate-descent Lasso, mini-batch SGD, convergence diagnostics, and validation. Implementations were tested against analytical solutions and/or scikit-learn where applicable.
<code>plot_generator.py</code>	2	Provided several individual plotting functions and modifications to existing plots. The functions were integrated into the existing plotting workflow and checked against the generated numerical results.

REFERENCES

- [1] University of Oslo, “Project 1 on machine learning: Regression analysis, resampling methods and gradient descent,” Jupyter notebook for FYS-STK3155/FYS4155, Data Analysis and Machine Learning (2026), published 31 August 2026.
- [2] M. Hjorth-Jensen, “Machine learning and the physical sciences: From discriminative methods to generative methods and reinforcement learning,” (2026), course manuscript, Chapter 3: Linear Regression.
- [3] M. Hjorth-Jensen, “Machine learning and the physical sciences: From discriminative methods to generative methods and reinforcement learning,” (2026), course manuscript, Chapter 2: Elements of Probability Theory and Statistics.
- [4] M. Hjorth-Jensen, “Machine learning and the physical sciences: From discriminative methods to generative methods and reinforcement learning,” (2026), course manuscript, Chapter 4: Optimization.
- [5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, *Journal of Machine Learning Research* **12**, 2825 (2011).
- [6] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, “JAX: Composable transformations of Python+NumPy programs,” (2018).
- [7] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, Springer Series in Statistics (Springer, New York, 2001).